CODEQUEST — ESPECIFICACIÓN DE NIVELES (Módulo de Retos)

Prompt para Cursor: los 4 niveles de programación por bloques

> *Pega este documento en Cursor junto con el prompt general de CodeQuest que ya tienes. Este archivo se enfoca **solo** en el módulo de retos del estudiante: la mecánica de juego, los 4 niveles, los mapas, los bloques y el motor que los ejecuta — inspirado en el mismo motor de bloques de CodeMonkey (Game Builder) que enviaste en las capturas, adaptado a TEA.*

---

1. CONCEPTO GENERAL DEL JUEGO

Un personaje (mascota de CodeQuest, ej. un robot o animal amigable, personalizable en la sección 8) se mueve por un **camino en cuadrícula (grid) de una sola fila**, de izquierda a derecha, hasta llegar a la meta (bandera 🏁) recogiendo estrellas ⭐ en el camino. El estudiante arma un programa con bloques que se ejecutan **en orden, de arriba hacia abajo**, al presionar "¡Correr!".

Esto es **exactamente el mismo modelo mental** que ya trabajaste en el motor adaptativo del documento general: cada ejecución de un reto es un "intento" (`attempt`) que se guarda con aciertos, número de intentos, tiempo de resolución y secuencia de bloques usada.

Tipos de casilla (tiles)

Símbolo	Nombre	Comportamiento
`.`	Camino	El personaje puede avanzar libremente
`M`	Inicio	Posición inicial del personaje
`_`	Hueco / pozo	Si el personaje avanza sobre esta casilla sin saltar, **cae y pierde el intento** (feedback suave, no punitivo)
`#`	Pared / obstáculo	Si el personaje avanza contra esta casilla sin saltar, **se detiene y pierde el intento**
`★`	Estrella	Objetivo opcional/bonus — se recoge automáticamente al pasar por encima
`F`	Meta	Bandera final — llegar aquí con éxito completa el reto

Regla de oro de las 4 niveles

Cada nivel introduce **una sola idea nueva** de pensamiento lógico y la refuerza con 4 a 6 retos de dificultad creciente. Nunca se introducen dos conceptos nuevos en el mismo nivel (principio de predictibilidad, ver documento de diseño general).

---

2. ARQUITECTURA TÉCNICA SUGERIDA

- **Motor de bloques:** Blockly (igual que se recomendó en el documento general). Se crean bloques personalizados (sección 6).
- **Motor de ejecución (interprete):** un servicio independiente `gridEngine.ts` que recibe el árbol de bloques ya serializado y el mapa del nivel, y devuelve un resultado paso a paso (para poder animar al personaje moviéndose bloque por bloque, no solo el resultado final).
- **Flujo de un intento:**
  1. Estudiante arma el programa con bloques.
  2. Presiona "¡Correr!".
  3. `gridEngine` ejecuta el programa instrucción por instrucción sobre el mapa del nivel.
  4. Se anima al personaje en tiempo real (velocidad ajustable, nunca más rápido de lo que un niño puede seguir visualmente).
  5. Al terminar (llega a la meta, cae en un hueco, choca con una pared, o se acaban los bloques sin llegar a la meta) se genera un registro:

```
{
  "estudianteId": "xxxx",
  "challengeId": "nivel_1_reto_3",
  "fechaHora": "2026-09-16T10:32:00Z",
  "aciertos": true,
  "numeroIntentos": 2,
  "tiempoResolucionSeg": 47,
  "secuenciaBloquesUsada": ["avanzar(3)", "avanzar(4)"]
}
```

Este registro es el mismo que ya definiste en la colección `attempts` de Firestore — aquí solo se especifica exactamente qué debe contener `secuenciaBloquesUsada` para este módulo: un arreglo de strings con cada bloque de nivel superior en el orden en que aparece, incluyendo su parámetro.

- **Motor adaptativo:** usa estos mismos campos (aciertos, intentos, tiempo) para decidir si el estudiante avanza, repite una variante, o recibe una pista extra — tal como se definió en la sección 4.2 del documento general.

---

3. PALETA DE COLORES DE BLOQUES (para mantener el mismo lenguaje visual de las capturas)

Categoría	Color aproximado (hex)	Bloques que contiene en CodeQuest
Movimiento	`#8A9A4B` (verde oliva)	Avanzar, Saltar
Control	`#4A9C7D` (verde azulado)	Repetir, Si, Si-Sino
Lógica y datos	`#A68A5A` (marrón cálido)	Y, O, No, ¿Hay hueco adelante?, ¿Hay pared adelante?, ¿Hay estrella cerca?
Variables	`#3FA6A6` (turquesa)	Crear variable, Cambiar variable, Obtener valor
Mostrar	`#4A4A8A` (índigo)	Mostrar mensaje, Ocultar mensaje
Eventos	`#9C3D3D` (opcional, uso futuro)	— no se usa en estos 4 niveles, reservado para niveles avanzados futuros

Usa colores suaves/pasteles de esta misma familia tonal (no los satures al 100%) para cumplir el principio de “colorido pero controlado”.

---

4. LOS 4 NIVELES

NIVEL 1 — “Primeros Pasos” (Secuenciación)

Concepto de pensamiento lógico: el computador ejecuta instrucciones exactamente en el orden en que se colocan; hay que planear la ruta completa antes de correr el programa.

Bloques disponibles: `Avanzar(n)` únicamente (se introduce `Saltar(n)` a partir del reto 4).

Bloque nuevo introducido: `Avanzar`.

Retos del Nivel 1

Reto 1.1

```
M . . F
```

Instrucción: “Arrastra un bloque Avanzar para llevar al personaje hasta la bandera.” Solución: `avanzar(3)`

Reto 1.2

```
M . . . . F
```

Solución: `avanzar(5)` Objetivo: practicar contar casillas más largas.

Reto 1.3 (introduce encadenar 2 bloques)

```
M . . ★ . . F
```

Instrucción: “Puedes usar más de un bloque Avanzar, uno después del otro.” Solución: `avanzar(3)`, `avanzar(3)` (recoge la estrella en el camino, bonus no obligatorio)

Reto 1.4 (introduce `Saltar`)

```
M . _ . . F
```

Instrucción: “Hay un hueco en el camino. Usa el bloque Saltar para pasarlo sin caer.” Solución: `avanzar(1)`, `saltar(1)`, `avanzar(2)`

Reto 1.5

```
M . _ . . . _ . . F
```

Solución: `avanzar(1)` , `saltar(1)` , `avanzar(2)` , `saltar(1)` , `avanzar(2)`

Reto 1.6 (cierre de nivel)

```
M . . _ . . . _ . F
```

Instrucción: “Planea todo tu camino antes de correr el programa.” (refuerza secuenciación completa) Solución: `avanzar(2)` , `saltar(1)` , `avanzar(3)` , `saltar(1)` , `avanzar(1)`

Criterio de aprobación del nivel: completar los 6 retos (con reintentos ilimitados). Al finalizar: insignia “Explorador Secuencial” 🐭.

NIVEL 2 — “El Poder de Repetir” (Bucles)

Concepto de pensamiento lógico: cuando una acción se repite muchas veces de la misma forma, se puede resumir con un bucle en lugar de escribir cada paso — reconocimiento de patrones.

Bloques disponibles: `Avanzar` , `Saltar` , `Repetir(n) { ... }` .

Bloque nuevo introducido: `Repetir` .

Retos del Nivel 2

Reto 2.1

```
M . . . . . . F
```

Instrucción: “Este camino es largo, pero todas las casillas son iguales. En vez de usar muchos bloques Avanzar, usa un solo bloque Repetir.” Límite sugerido: máximo 2 bloques en el área de código (obliga a usar el bucle). Solución: `repetir(6) { avanzar(1) }`

Reto 2.2 (patrón compuesto dentro del bucle)

```
M . _ . _ . _ . F
```

Instrucción: “Este camino repite el mismo patrón: avanzar y saltar. Repítelo con un solo bucle.” Solución: `repetir(3) { avanzar(1); saltar(1) }` , `avanzar(1)`

Reto 2.3 (recolección dentro de un bucle)

```
M ★ . ★ . ★ . ★ F
```

Solución: `repetir(4) { avanzar(2) }` Objetivo: mostrar que el bucle también sirve para recoger varias estrellas seguidas.

Reto 2.4 (dos bucles distintos en secuencia)

```
M . . . . _ . _ . F
```

Solución: `repetir(4) { avanzar(1) }` , `repetir(2) { saltar(1); avanzar(1) }` Objetivo: enseñar que se pueden usar varios bloques Repetir, uno después de otro, para distintos tramos del camino.

Reto 2.5 (cierre de nivel)

```
M . . . . . . . . . . F
```

Instrucción: “Encuentra tú mismo el patrón que se repite y arma el bucle más corto posible.” Solución esperada: `repetir(11) { avanzar(1) }` Este reto no da pistas de conteo; el estudiante debe generalizar por sí mismo (evaluación de transferencia del concepto).

Criterio de aprobación: completar los 5 retos. Insignia: “Maestro de los Bucles” 🔁.

NIVEL 3 — “Toma de Decisiones” (Condicionales)

Concepto de pensamiento lógico: el programa puede “preguntar” algo y decidir qué hacer según la respuesta (si es verdadero, haz A; si es falso, haz B).

Bloques disponibles: todos los anteriores + `Si { ... }` , `Si-Sino { ... } { ... }` , y los sensores: `¿Hay hueco adelante?` , `¿Hay pared adelante?` , `¿Hay estrella adelante?` .

Bloques nuevos introducidos: `Si` , `Si-Sino` , sensores.

Retos del Nivel 3

Reto 3.1 (introducción aislada, sin bucle)

```
M . _ . F
```

Instrucción: “En vez de decidir tú si saltar, dile al programa que decida: si hay un hueco adelante, que salte; si no, que avance.” Solución: `si (¿hay hueco adelante?) { saltar(1) } sino { avanzar(1) }`, repetido manualmente 3 veces en secuencia (sin bucle todavía, para que el foco sea 100% el condicional).

Reto 3.2 (combina con bucle — el gran salto conceptual)

```
M . _ . . _ . . . F
```

Instrucción: “Ahora combina lo que aprendiste: usa un bucle que se repita, y adentro coloca la decisión Si/Sino.” Solución:

```
repetir(9) {
  si (¿hay hueco adelante?) {
    saltar(1)
  } sino {
    avanzar(1)
  }
}
```

Este es el reto más importante del nivel: un solo programa corto que funciona sin importar dónde estén los huecos.

Reto 3.3 (introduce pared, requiere Si-Sino con dos condiciones distintas)

```
M . _ . # . . F
```

Solución:

```
repetir(7) {
  si (¿hay hueco adelante?) {
    saltar(1)
  } sino si (¿hay pared adelante?) {
    saltar(1)
  } sino {
    avanzar(1)
  }
}
```

(Nota para Cursor: si el motor de bloques no soporta “sino-si” encadenado, usar dos bloques `Si` anidados dentro del `Sino` del primero.)

Reto 3.4 (condicional + retroalimentación visual)

```
M . ★ . _ . F
```

Instrucción: añade un bloque `Mostrar mensaje` cuando se recoge una estrella. Solución:

```
repetir(6) {
  si (¿hay estrella adelante?) {
    avanzar(1); mostrar("¡Genial!")
  } sino si (¿hay hueco adelante?) {
    saltar(1)
  } sino {
    avanzar(1)
  }
}
```

Reto 3.5 (cierre de nivel — “algoritmo general”)

```
M . _ . # . . _ . . F
```

Instrucción: “Este es el camino más largo hasta ahora. Tu programa debe funcionar sin que tengas que mirar el mapa: deja que el Si/Sino decida por ti.” Misma solución patrón que 3.3, aplicada a un camino más largo — refuerza que **un mismo algoritmo corto resuelve caminos distintos**.

Criterio de aprobación: completar los 5 retos. Insignia: “Estratega Lógico” 🧠.

NIVEL 4 — “Desafío Final: Lógica Combinada”

Concepto de pensamiento lógico: generalización de un algoritmo (que funcione en cualquier variante del problema), operadores lógicos compuestos (Y / O / No), y conteo con variables.

Bloques disponibles: todos los anteriores + `Y`, `O`, `No`, `Crear variable`, `Cambiar variable`, widget `Contador`.

Bloques nuevos introducidos: operadores lógicos, variables/contador.

Retos del Nivel 4

Reto 4.1 — Mapa aleatorio (examen de generalización) El mapa se genera aleatoriamente cada vez que se juega este reto (usar bloque `Random` en el generador de niveles, no en el código del estudiante): largo fijo de 10 casillas, con 0 a 3 huecos y 0 a 2 paredes en posiciones aleatorias. Instrucción: "Este camino cambia cada vez que lo intentas. Si tu programa del Nivel 3 estaba bien hecho, ¡funcionará igual de bien aquí!" Solución esperada: el mismo patrón de bucle + Si/Sino/Sino de los retos 3.2/3.3, ahora puesto a prueba contra un mapa que el estudiante no puede memorizar.

Reto 4.2 — Contador de estrellas

```
M ★ . _ ★ . # ★ . F
```

Instrucción: "Cada vez que recojas una estrella, tu contador debe subir en 1. Al final debes tener 3 estrellas." Solución:

```
crear variable "puntaje" = 0
repetir(9) {
  si (¿hay estrella adelante?) {
    avanzar(1); cambiar variable "puntaje" + 1
  } sino si (¿hay hueco adelante?) {
    saltar(1)
  } sino si (¿hay pared adelante?) {
    saltar(1)
  } sino {
    avanzar(1)
  }
}
mostrar contador "puntaje"
```

Reto 4.3 — Operadores lógicos compuestos

```
M . ★ _ . ★ # . F
```

Instrucción: "A veces hay una estrella justo antes de un hueco o una pared. Usa Y / O para decidir con más precisión." Ejemplo de condición compuesta: `si ( (¿hay estrella adelante?) Y (no ¿hay hueco adelante?) ) { avanzar(1) } sino { saltar(1) }`

Reto 4.4 — Reto maestro de cierre de plataforma Mapa aleatorio de 15 a 20 casillas, combinando huecos, paredes y estrellas en cualquier proporción, generado de nuevo en cada intento. Instrucción: "Este es el reto final de CodeQuest. Escribe un solo programa capaz de resolver cualquier camino que aparezca." Al completarlo: pantalla de celebración (animación suave, sin parpadeos ni sonidos fuertes) + insignia final "¡Programador CodeQuest!" 🏆 + mensaje motivador para el estudiante y notificación automática al portal de padres y al panel del profesional.

Criterio de aprobación: completar los 4 retos. Este nivel marca el cierre del recorrido de retos del MVP.

---

5. TABLA RESUMEN DE PROGRESIÓN

Nivel	Concepto	N° retos	Bloques nuevos	Insignia
1. Primeros Pasos	Secuenciación	6	Avanzar, Saltar	Explorador Secuencial
2. El Poder de Repetir	Bucles	5	Repetir	Maestro de los Bucles
3. Toma de Decisiones	Condicionales	5	Si, Si-Sino, sensores	Estratega Lógico
4. Desafío Final	Lógica combinada + variables	4	Y/O/No, Variables, Contador	Programador CodeQuest

Total: 20 retos distribuidos en 4 niveles — suficiente para varias semanas de uso real sin agotar el contenido, y con margen para que el motor adaptativo repita variantes de un mismo reto cuando el estudiante lo necesite (ver sección 4.2 del documento general: si el desempeño es bajo, el sistema puede regenerar un mapa similar en vez de avanzar).

---

6. DEFINICIONES DE BLOQUES PARA BLOCKLY (JSON listo para Cursor)

```
[
  {
    "type": "avanzar",
```

```
  "message0": "Avanzar %1",
  "args0": [{ "type": "field_number", "name": "PASOS", "value": 1, "min": 1, "max": 10 }],
  "previousStatement": null,
  "nextStatement": null,
  "colour": "#8A9A4B",
  "tooltip": "Mueve al personaje hacia adelante el número de casillas indicado."
},
{
  "type": "saltar",
  "message0": "Saltar %1",
  "args0": [{ "type": "field_number", "name": "PASOS", "value": 1, "min": 1, "max": 3 }],
  "previousStatement": null,
  "nextStatement": null,
  "colour": "#8A9A4B",
  "tooltip": "El personaje salta hacia adelante, pasando por encima de huecos o paredes bajas."
},
{
  "type": "repetir",
  "message0": "Repetir %1 veces %2 %3",
  "args0": [
    { "type": "field_number", "name": "VECES", "value": 2, "min": 1  "max": 20 },
    { "type": "input_dummy" },
    { "type": "input_statement", "name": "HACER" }
  ],
  "previousStatement": null,
  "nextStatement": null,
  "colour": "#4A9C7D",
  "tooltip": "Repite las instrucciones de adentro el número de veces indicado."
},
{
  "type": "si",
  "message0": "Si %1 entonces %2 %3",
  "args0": [
    { "type": "input_value", "name": "CONDICION", "check": "Boolean" },
    { "type": "input_dummy" },
    { "type": "input_statement", "name": "HACER" }
  ],
  "previousStatement": null,
  "nextStatement": null,
  "colour": "#4A9C7D",
  "tooltip": "Si la condición es verdadera, ejecuta las instrucciones de adentro."
},
{
  "type": "si_sino",
  "message0": "Si %1 entonces %2 %3 si no %4 %5",
  "args0": [
    { "type": "input_value", "name": "CONDICION", "check": "Boolean" },
    { "type": "input_dummy" },
    { "type": "input_statement", "name": "HACER_SI" },
    { "type": "input_dummy" },
    { "type": "input_statement", "name": "HACER_SINO" }
  ],
  "previousStatement": null,
  "nextStatement": null,
  "colour": "#4A9C7D",
  "tooltip": "Si la condición es verdadera ejecuta la primera parte; si es falsa, ejecuta la segunda."
},
{
  "type": "sensor_hueco_adelante",
  "message0": "¿Hay hueco adelante?",
  "output": "Boolean",
  "colour": "#A68A5A",
  "tooltip": "Devuelve verdadero si la siguiente casilla es un hueco."
},
{
  "type": "sensor_pared_adelante",
  "message0": "¿Hay pared adelante?",
  "output": "Boolean",
  "colour": "#A68A5A",
  "tooltip": "Devuelve verdadero si la siguiente casilla es una pared."
},
{
  "type": "sensor_estrella_adelante",
  "message0": "¿Hay estrella adelante?",
  "output": "Boolean",
  "colour": "#A68A5A",
  "tooltip": "Devuelve verdadero si la siguiente casilla tiene una estrella."
},
{
  "type": "logica_y",
  "message0": "%1 Y %2",
  "args0": [
    { "type": "input_value", "name": "A", "check": "Boolean" },
    { "type": "input_value", "name": "B", "check": "Boolean" }
  ],
  "output": "Boolean",
  "colour": "#A68A5A"
},
{
```

```
    "type": "logica_o",
    "message0": "%1 O %2",
    "args0": [
      { "type": "input_value", "name": "A", "check": "Boolean" },
      { "type": "input_value", "name": "B", "check": "Boolean" }
    ],
    "output": "Boolean",
    "colour": "#A68A5A"
  },
  {
    "type": "logica_no",
    "message0": "No %1",
    "args0": [{ "type": "input_value", "name": "A", "check": "Boolean" }],
    "output": "Boolean",
    "colour": "#A68A5A"
  },
  {
    "type": "crear_variable_contador",
    "message0": "crear variable %1 = %2",
    "args0": [
      { "type": "field_input", "name": "NOMBRE", "text": "puntaje" },
      { "type": "field_number", "name": "VALOR_INICIAL", "value": 0 }
    ],
    "previousStatement": null,
    "nextStatement": null,
    "colour": "#3FA6A6"
  },
  {
    "type": "cambiar_variable",
    "message0": "cambiar %1 en %2",
    "args0": [
      { "type": "field_input", "name": "NOMBRE", "text": "puntaje" },
      { "type": "field_number", "name": "CANTIDAD", "value": 1 }
    ],
    "previousStatement": null,
    "nextStatement": null,
    "colour": "#3FA6A6"
  },
  {
    "type": "mostrar_mensaje",
    "message0": "mostrar mensaje %1",
    "args0": [{ "type": "field_input", "name": "TEXTO", "text": "¡Genial!" }],
    "previousStatement": null,
    "nextStatement": null,
    "colour": "#4A4A8A"
  }
]
```

7. ESQUEMA JSON DE UN NIVEL COMPLETO (ejemplo: Nivel 1)

```
{
  "levelId": "nivel_1_secuenciacion",
  "levelName": "Primeros Pasos",
  "concepto": "secuenciacion",
  "bloquesDisponibles": ["avanzar", "saltar"],
  "insigniaAlCompletar": "Explorador Secuencial",
  "retos": [
    {
      "retoId": "1.1",
      "grid": ["M", ".", ".", "F"],
      "instruccion": "Arrastra un bloque Avanzar para llevar al personaje hasta la bandera.",
      "solucionEjemplo": ["avanzar(3)"]
    },
    {
      "retoId": "1.2",
      "grid": ["M", ".", ".", ".", ".", "F"],
      "instruccion": "Cuenta cuántas casillas hay hasta la bandera.",
      "solucionEjemplo": ["avanzar(5)"]
    },
    {
      "retoId": "1.3",
      "grid": ["M", ".", ".", "*", ".", ".", "F"],
      "instruccion": "Puedes usar más de un bloque Avanzar, uno después del otro.",
      "solucionEjemplo": ["avanzar(3)", "avanzar(3)"]
    },
    {
      "retoId": "1.4",
      "grid": ["M", ".", "_", ".", ".", "F"],
      "instruccion": "Hay un hueco en el camino. Usa el bloque Saltar para pasarlo sin caer.",
      "solucionEjemplo": ["avanzar(1)", "saltar(1)", "avanzar(2)"]
    },
    {
      "retoId": "1.5",
```

```
      "grid": ["M", ".", "_", ".", ".", "_", ".", ".", "F"],
      "instruccion": "Este camino tiene dos huecos.",
      "solucionEjemplo": ["avanzar(1)", "saltar(1)", "avanzar(2)", "saltar(1)", "avanzar(2)"]
    },
    {
      "retoId": "1.6",
      "grid": ["M", ".", ".", "_", ".", ".", ".", "_", ".", "F"],
      "instruccion": "Planea todo tu camino antes de correr el programa.",
      "solucionEjemplo": ["avanzar(2)", "saltar(1)", "avanzar(3)", "saltar(1)", "avanzar(1)"]
    }
  ]
}
```

(Cursor: replica esta misma estructura para `nivel_2_bucles.json`, `nivel_3_condicionales.json` y `nivel_4_logica_combinada.json` usando los mapas y soluciones de la sección 4. Para el Nivel 4, el campo `grid` de los retos 4.1 y 4.4 debe generarse dinámicamente con una función `generarMapaAleatorio(largo, nHuecos, nParedes, nEstrellas)` en vez de estar fijo en el JSON.)

---

8. PERSONAJE Y ESTÉTICA DEL JUEGO

- **Mascota:** un personaje propio de CodeQuest (no usar el mono de CodeMonkey por temas de identidad de marca) — sugerido: un robot pequeño y amigable, o un animal redondeado (ej. un zorro o un koala), con expresiones faciales simples y claras (feliz, concentrado, "casi lo logras"), nunca expresiones de miedo, dolor o frustración intensa ante un error.
- **Fondo:** cielo despejado con nubes suaves, camino de piedra o césped en tonos de la paleta ya definida (ver documento general, sección 2).
- **Feedback de acierto:** el personaje celebra con una animación corta (salto feliz), sonido opcional suave tipo "campanita", y aparece el mensaje "¡Lo lograste!" con una estrella o insignia.
- **Feedback de error (caer en hueco / chocar con pared):** el personaje simplemente se detiene con una expresión de "ups" (nunca "se cae dramáticamente" ni sonido de fallo fuerte); mensaje calmado: "Casi lo tienes, ¡inténtalo de nuevo!". Reinicio inmediato del reto sin penalización visible.
- **Indicador de progreso:** igual que en la captura que enviaste ("Ejercicio 1 de 10"), mostrar siempre "Reto X de Y" en la parte superior, y un mapa de niveles tipo camino (sección 4.6 del documento general) para ver el avance global.

---

9. INSTRUCCIÓN FINAL PARA CURSOR

> *"Implementa el módulo de retos usando Blockly con los bloques personalizados de la sección 6, un motor de ejecución (`gridEngine.ts`) que interprete el programa contra el mapa de cada reto según las reglas de la sección 1, y carga los 4 niveles con los retos exactos de la sección 4 (usa los JSON de la sección 7 como base y complétalos para los niveles 2, 3 y 4). Cada intento debe generar un registro compatible con la colección `attempts` ya definida en el documento general de CodeQuest, para alimentar el motor adaptativo y los paneles de padres/profesionales."*

Documento complementario al "CodeQuest_Especificacion_para_Cursor" — mismo proyecto de investigación, Universidad Libre Seccional Cúcuta, Ingeniería en TIC.